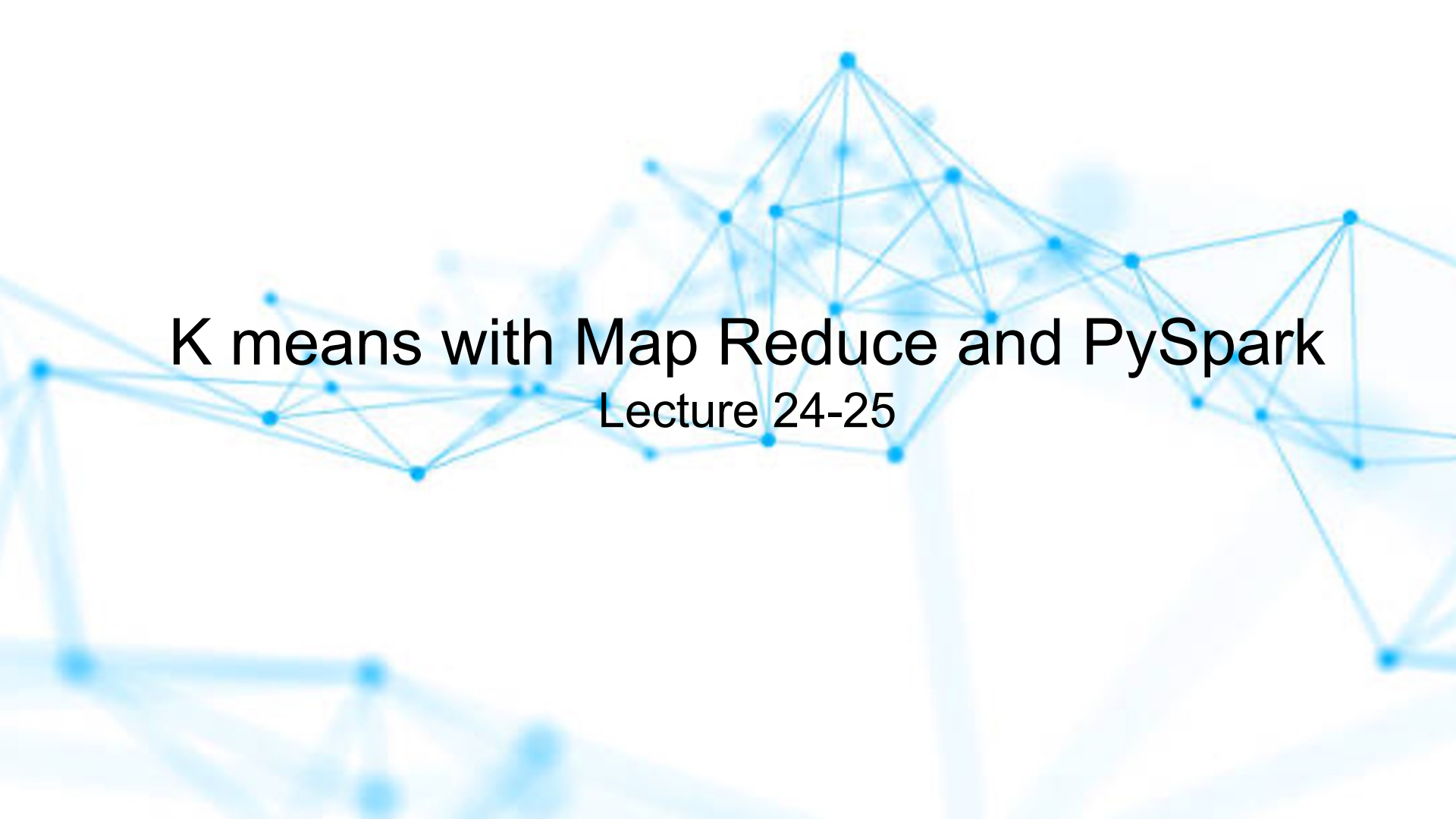




K means with Map Reduce and PySpark

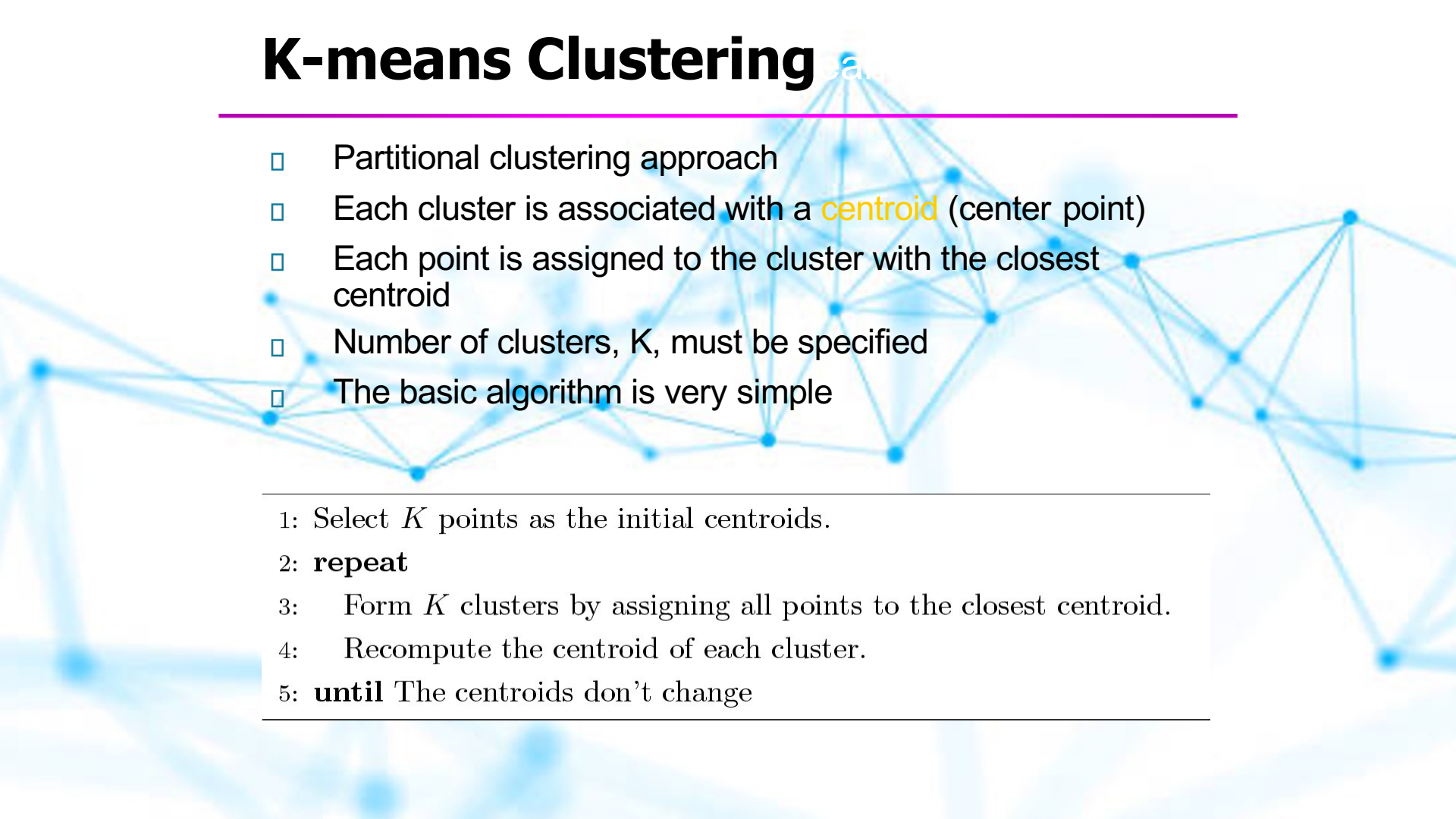
Lecture 24-25

Quiz 6

Wednesday 17 May, 2023
Lecture 21-28



K-means Clustering



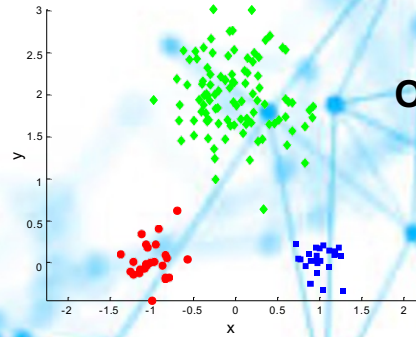
- Partitional clustering approach
- Each cluster is associated with a **centroid** (center point)
- Each point is assigned to the cluster with the closest centroid
- Number of clusters, K , must be specified
- The basic algorithm is very simple

-
- 1: Select K points as the initial centroids.
 - 2: **repeat**
 - 3: Form K clusters by assigning all points to the closest centroid.
 - 4: Recompute the centroid of each cluster.
 - 5: **until** The centroids don't change
-

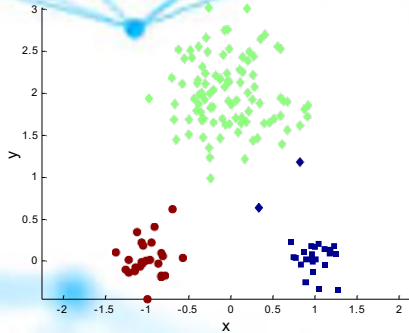
K-means Clustering

- Initial centroids are often chosen randomly.
 - Clusters produced vary from one run to another.
- The centroid is (typically) the mean of the points in the cluster.
- 'Closeness' is measured by Euclidean distance, cosine similarity, correlation, etc.
- K-means will converge for common similarity measures mentioned above.
- Most of the convergence happens in the first few iterations.
 - Often the stopping condition is changed to 'Until relatively few points change clusters'
- Complexity is $O(n * K * I * d)$
 - n = number of points, K = number of clusters,
 I = number of iterations, d = number of attributes

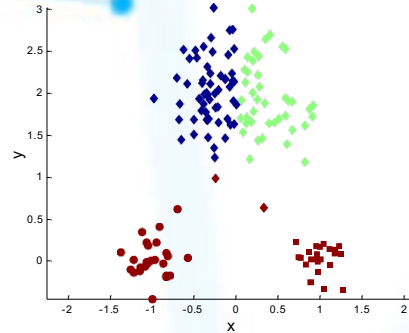
K-means Clustering



Original Points



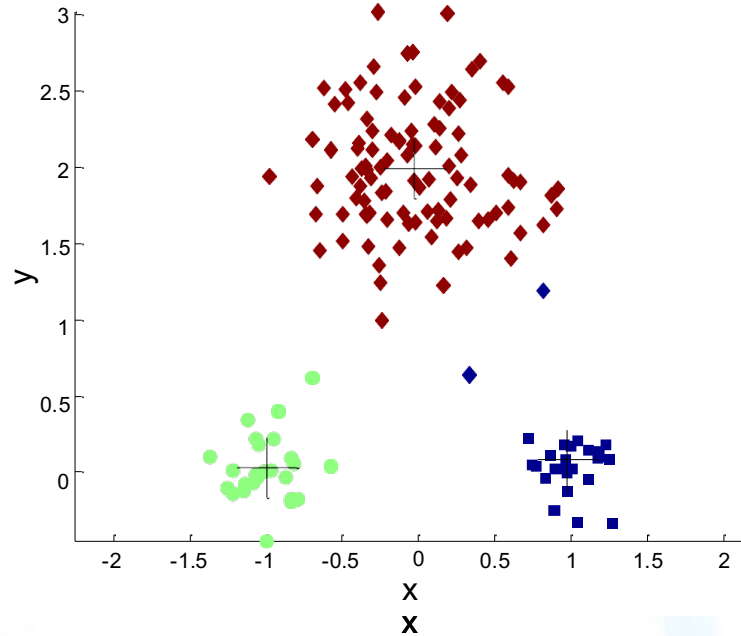
Optimal Clustering



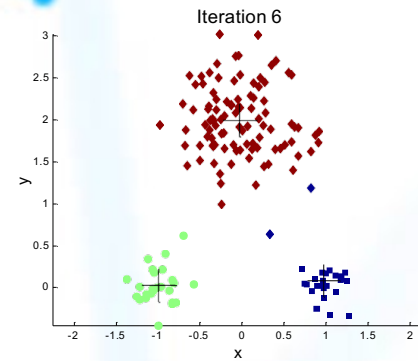
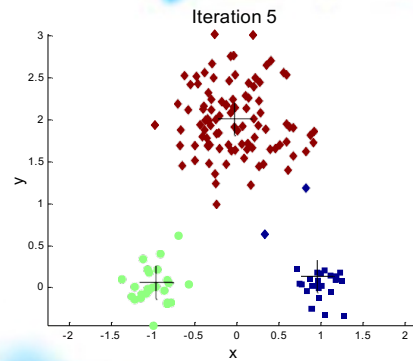
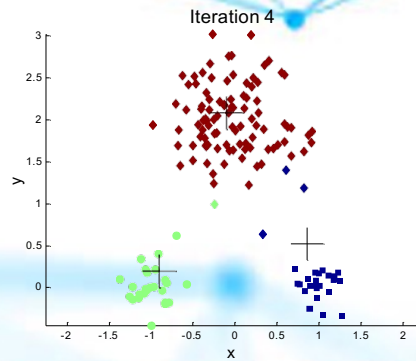
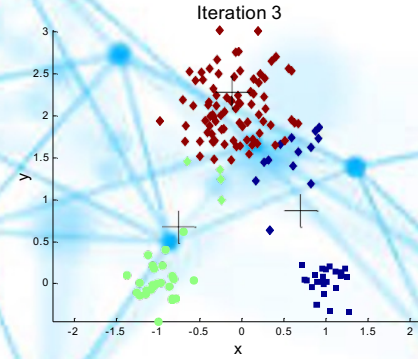
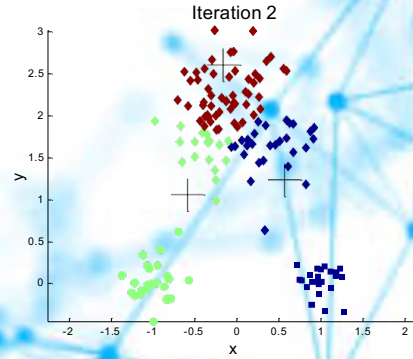
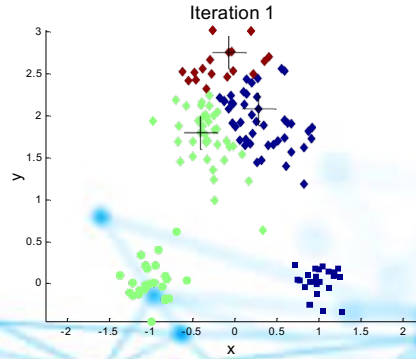
Sub-optimal Clustering

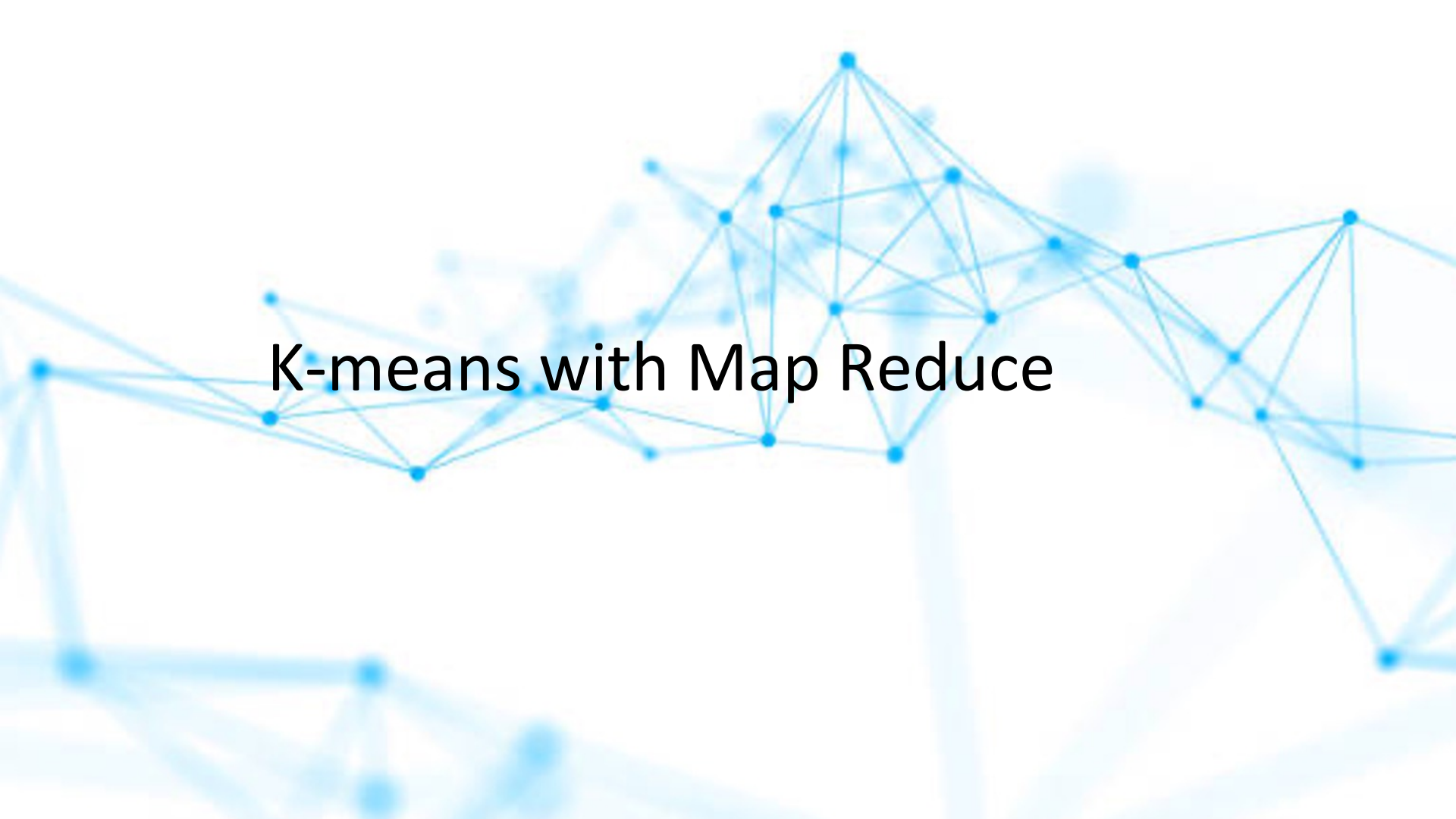
K-means Clustering

Iteration 6



K-means Clustering





K-means with Map Reduce

K-means with Map Reduce

Below is a proposed plan for your exercise:

centroids = k random sampled points from the dataset

Map:

- Given a point and the set of centroids
- Calculate the distance between the point and each centroid
- Emit the point and the closest centroid

Reduce:

- Given the centroid and the points belonging to its cluster
- Calculate the new centroid as the arithmetic mean position of the points
- Emit the new centroid

`prev_centroids = centroids`

`centroids = new_centroids`

`while prev_centroids - centroids > threshold`

The mapper class calculates the distance between the data point and each centroid. Then emits the index of the closest centroid and the data point:

```
class MAPPER
method MAP(file_offset, point)
    min_distance = POSITIVE_INFINITY
    closest_centroid = -1
    for all centroid in list_of_centroids
        distance = distance(centroid, point)
        if (distance < min_distance)
            closest_centroid = index_of(centroid)
            min_distance = distance
    EMIT(closest_centroid, point)
```

The reducer calculates the new approximation of the centroid and emits it.

```
class REDUCER
method REDUCER(centroid_index, list_of_point_sums)
    number_of_points = partial_sum.number_of_points
    point_sum = 0
    for all partial_sum in list_of_partial_sums:
        point_sum += partial_sum
        point_sum.number_of_points += partial_sum.number_of_points
    centroid_value = point_sum / point_sum.number_of_points
    EMIT(centroid_index, centroid_value)
```

```

from mrjob.job import MRJob
import numpy as np

class KMeansMR(MRJob):

    def mapper_init(self):
        self.centroids = np.array([[2.93, 0.72, 1.77, 0.71],
                                    [6.85, 3.09, 5.39, 2.03],
                                    [5.01, 3.42, 1.46, 0.25]])

    def mapper(self, _, line):
        point = np.array([float(x) for x in line.split()])
        distances = np.linalg.norm(self.centroids - point, axis=1)
        yield np.argmin(distances), (point, 1)

    def combiner(self, cluster_id, partial_counts):
        partial_sum = np.zeros(4)
        total_count = 0
        for point, count in partial_counts:
            partial_sum += point * count
            total_count += count
        yield cluster_id, (partial_sum, total_count)

```

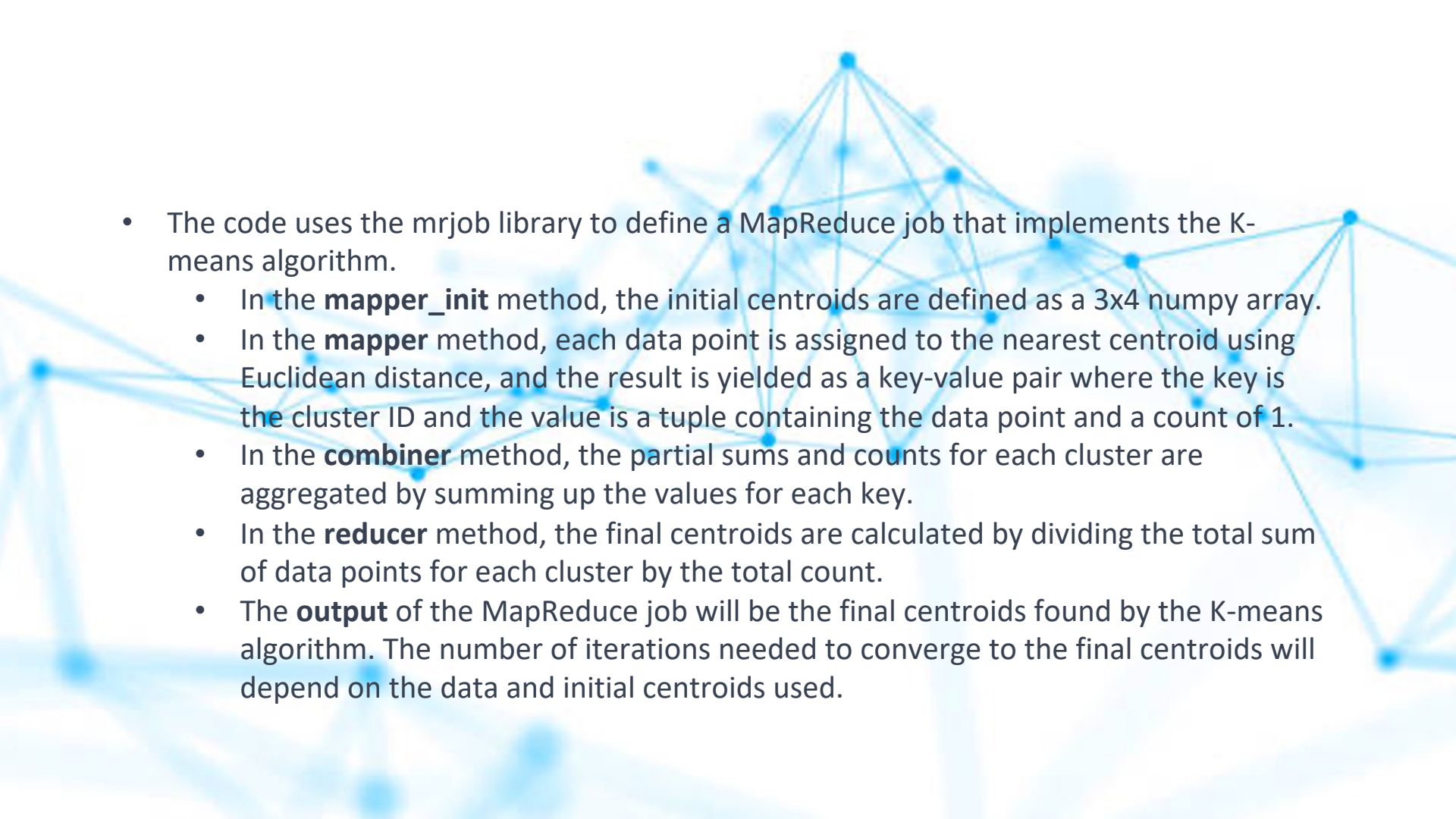


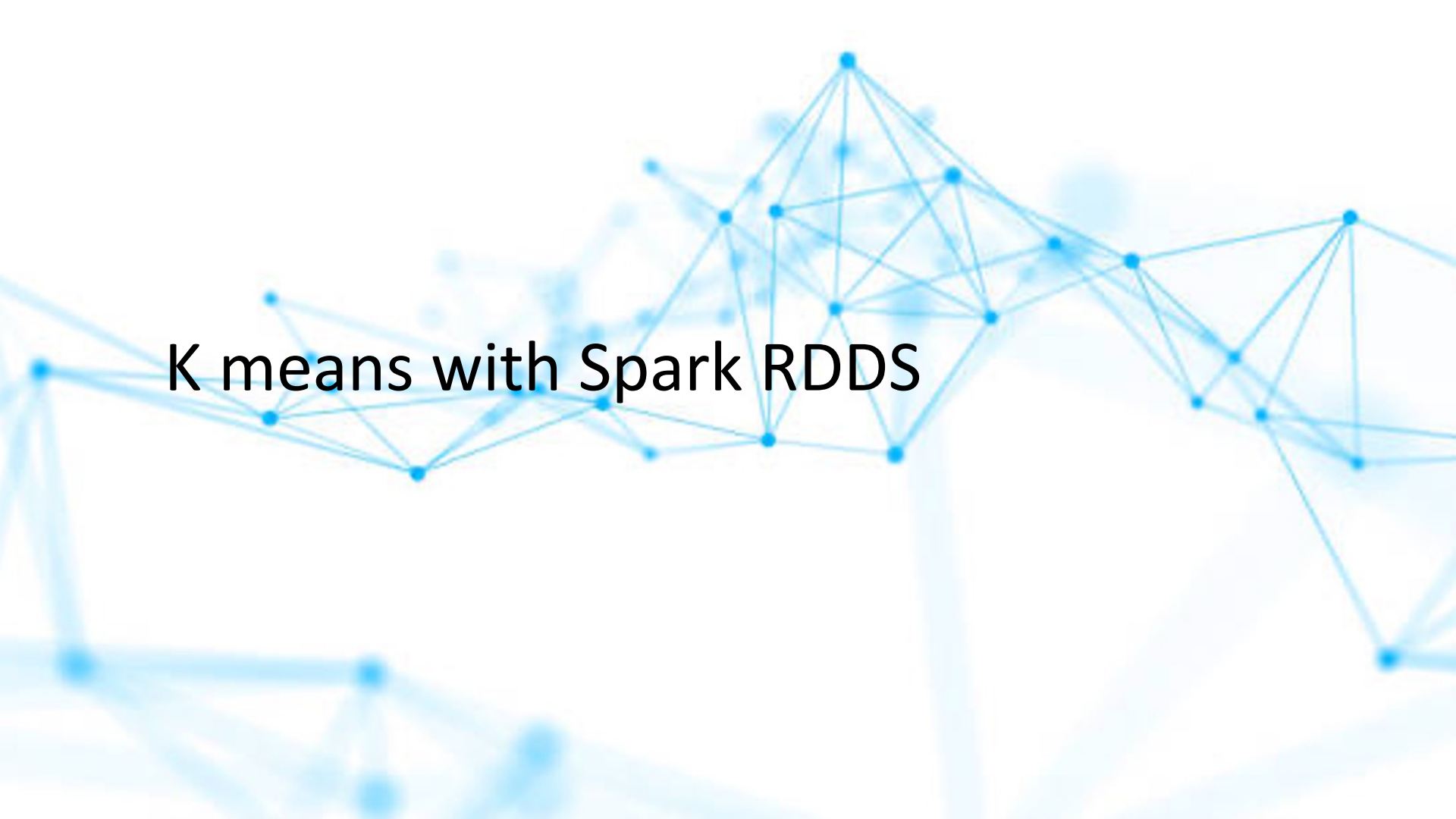
```

    def reducer(self, cluster_id, partial_sums):
        total_sum = np.zeros(4)
        total_count = 0
        for partial_sum, count in partial_sums:
            total_sum += partial_sum
            total_count += count
        yield cluster_id, total_sum / total_count

if __name__ == '__main__':
    KMeansMR.run()

```

- 
- The code uses the mrjob library to define a MapReduce job that implements the K-means algorithm.
 - In the **mapper_init** method, the initial centroids are defined as a 3x4 numpy array.
 - In the **mapper** method, each data point is assigned to the nearest centroid using Euclidean distance, and the result is yielded as a key-value pair where the key is the cluster ID and the value is a tuple containing the data point and a count of 1.
 - In the **combiner** method, the partial sums and counts for each cluster are aggregated by summing up the values for each key.
 - In the **reducer** method, the final centroids are calculated by dividing the total sum of data points for each cluster by the total count.
 - The **output** of the MapReduce job will be the final centroids found by the K-means algorithm. The number of iterations needed to converge to the final centroids will depend on the data and initial centroids used.

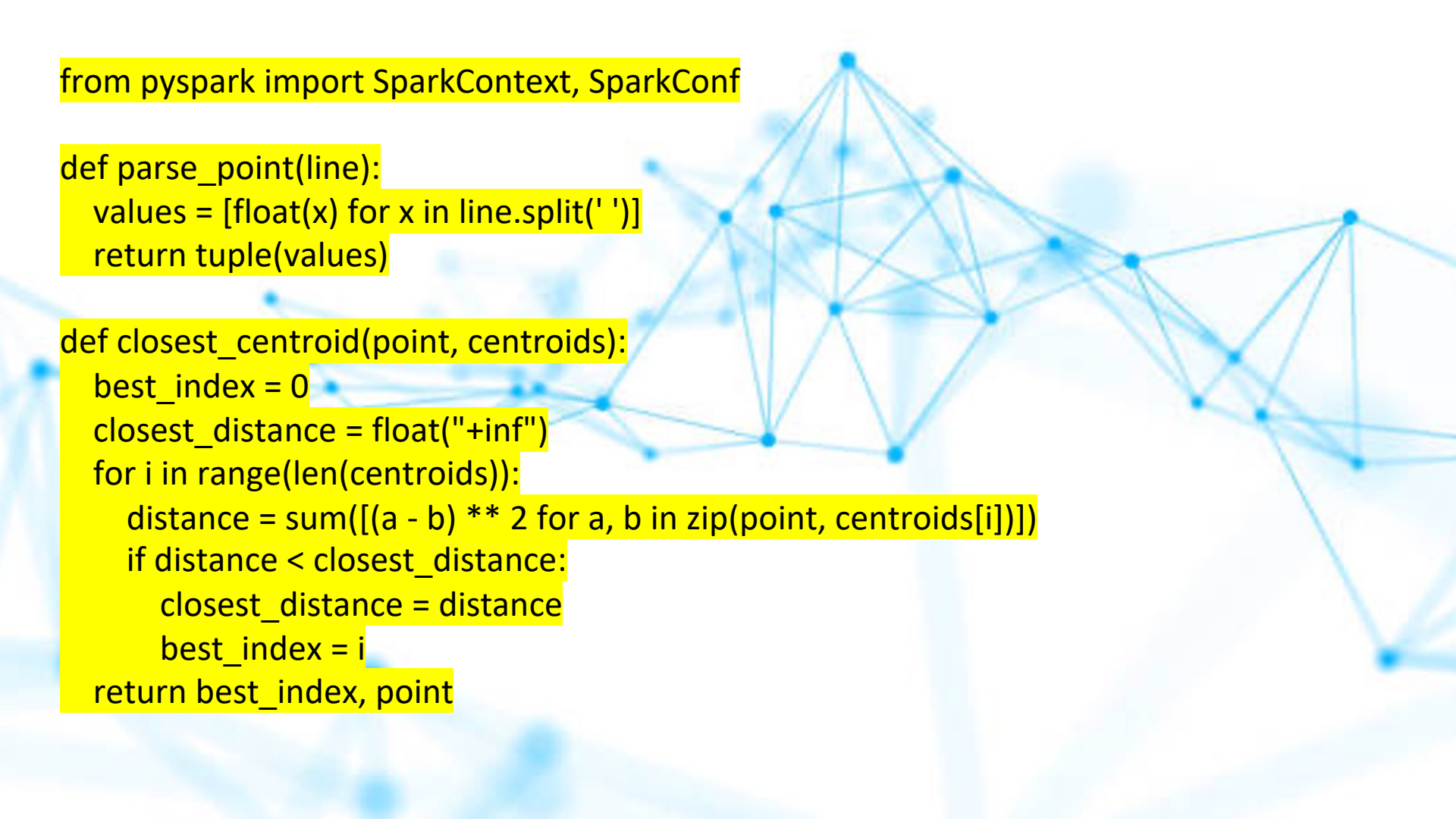


K means with Spark RDDS

```
from pyspark import SparkContext, SparkConf
```

```
def parse_point(line):  
    values = [float(x) for x in line.split(' ')]  
    return tuple(values)
```

```
def closest_centroid(point, centroids):  
    best_index = 0  
    closest_distance = float("+inf")  
    for i in range(len(centroids)):  
        distance = sum([(a - b) ** 2 for a, b in zip(point, centroids[i])])  
        if distance < closest_distance:  
            closest_distance = distance  
            best_index = i  
    return best_index, point
```



```
def compute_centroids(points):  
    num_points = len(points)  
    num_dims = len(points[0])  
    sum_point = [sum(x) for x in zip(*points)]  
    return [x / num_points for x in sum_point]
```

```
def kmeans(data, k, converge_dist):
```

```
    k_points = data.takeSample(False, k, 1)  
    temp_dist = 1.0
```

```
    while temp_dist > converge_dist:
```

```
        closest = data.map(lambda p: closest_centroid(p, k_points))
```

```
        point_stats = closest.groupByKey().mapValues(list)
```

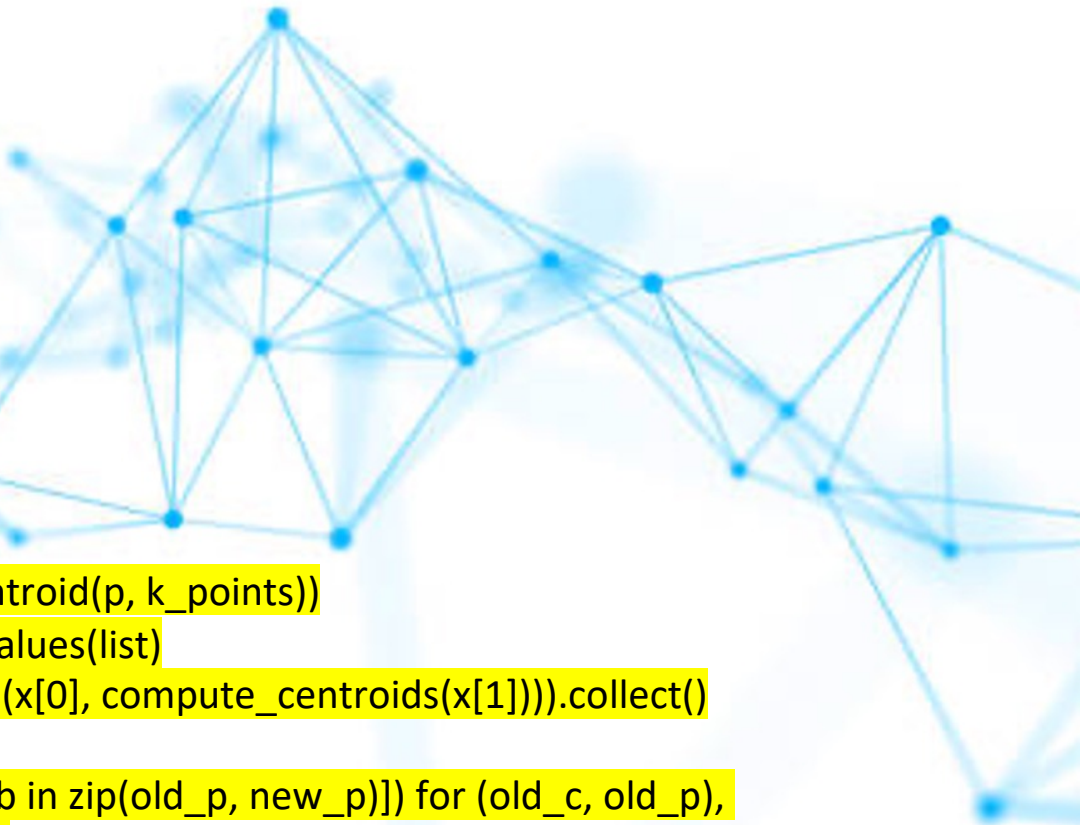
```
        new_points = point_stats.map(lambda x: (x[0], compute_centroids(x[1]))).collect()
```

```
        temp_dist = sum([sum([(a - b) ** 2 for a, b in zip(old_p, new_p)]) for (old_c, old_p),  
                        (new_c, new_p) in zip(k_points, new_points)])
```

```
        for (i, point) in new_points:
```

```
            k_points[i] = point
```

```
    return k_points
```



```
def compute_centroids(points):  
    num_points = len(points)  
    num_dims = len(points[0])  
    sum_point = [sum(x) for x in zip(*points)]  
    return [x / num_points for x in sum_point]
```

```
def kmeans(data, k, converge_dist):
```

```
    k_points = data.takeSample(False, k, 1)  
    temp_dist = 1.0
```

```
    while temp_dist > converge_dist:
```

```
        closest = data.map(lambda p: closest_centroid(p, k_points))
```

```
        point_stats = closest.groupByKey().mapValues(list)
```

```
        new_points = point_stats.map(lambda x: (x[0], compute_centroids(x[1]))).collect()
```

```
        temp_dist = sum([sum([(a - b) ** 2 for a, b in zip(old_p, new_p)]) for (old_c, old_p),  
                        (new_c, new_p) in zip(k_points, new_points)])
```

```
        for (i, point) in new_points:
```

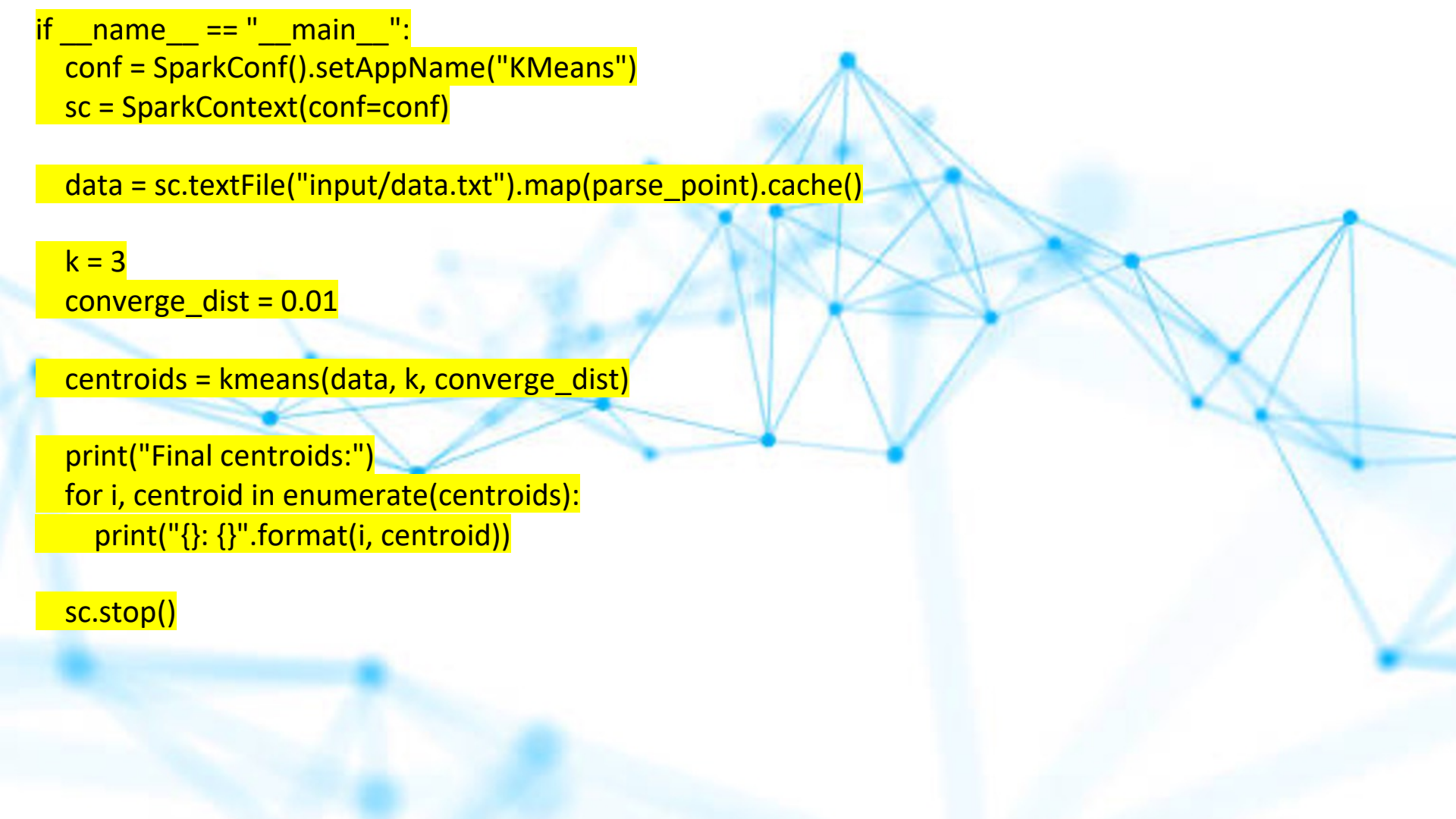
```
            k_points[i] = point
```

```
    return k_points
```

is used to calculate the sum of data points and the count of data points assigned to each centroid.

used to calculate the new centroids based on the data points assigned to each centroid.

calculates the sum of squared distances between the old and new centroid positions for each centroid.



```
if __name__ == "__main__":  
    conf = SparkConf().setAppName("KMeans")  
    sc = SparkContext(conf=conf)
```

```
data = sc.textFile("input/data.txt").map(parse_point).cache()
```

```
k = 3  
converge_dist = 0.01
```

```
centroids = kmeans(data, k, converge_dist)
```

```
print("Final centroids:")  
for i, centroid in enumerate(centroids):  
    print("{}: {}".format(i, centroid))
```

```
sc.stop()
```

```
points = sc.textFile(INPUT_PATH).map(Point).cache()
initial_centroids = init_centroids(points, k=parameters["k"])
```

```
def init_centroids(dataset, k):
    start_time = time.time()
    initial_centroids = dataset.takeSample(False, k)
    print("init centroid execution:", len(initial_centroids), "in",
          (time.time() - start_time), "s")
    return initial_centroids
```

```
while True:
    print("--Iteration n. {itr:d}".format(itr=n+1), end="\r",
          flush=True)
    cluster_assignment_rdd = points.map(assign_centroids)
    sum_rdd = cluster_assignment_rdd.reduceByKey(lambda x, y: x.sum(y))
    centroids_rdd = sum_rdd.mapValues(lambda x:
    x.get_average_point()).sortByKey(ascending=True)

    new_centroids = [item[1] for item in centroids_rdd.collect()]
    stop = stopping_criterion(new_centroids, parameters["threshold"])

    n += 1
    if(stop == False and n < parameters["maxiteration"]):
        centroids_broadcast = sc.broadcast(new_centroids)
    else:
        break
```

```
def stopping_criterion(new_centroids, threshold):
    old_centroids = centroids_broadcast.value
    for i in range(len(old_centroids)):
        check = old_centroids[i].distance(new_centroids[i],
        distance_broadcast.value) <= threshold
        if check == False:
            return False
    return True
```

```
def assign_centroids(p):
    min_dist = float("inf")
    centroids = centroids_broadcast.value
    nearest_centroid = 0
    for i in range(len(centroids)):
        distance = p.distance(centroids[i], distance_broadcast.value)
        if(distance < min_dist):
            min_dist = distance
            nearest_centroid = i
    return (nearest_centroid, p)
```

Resources to explore

- <https://github.com/seraogianluca/k-means-mapreduce>

Spark ML Library

The background of the slide features a complex, abstract network graph. It consists of numerous small blue circular nodes connected by thin, light blue lines. The nodes are distributed across the frame, with a higher density in the upper right and lower right areas, and a more sparse arrangement towards the left. The overall effect is a sense of interconnectedness and data flow, typical of machine learning or network science visualizations.

What is Spark MLlib?



MLlib stands for **Machine Learning library** in Spark. The goal of this library is to make practical machine learning scalable and easy to implement



Collects data



Processes data



Builds data model

What is Machine Learning?

Machine learning is the science of training computers to learn from data and perform specific tasks without being explicitly programmed



Machine Learning Algorithms



Machine Learning Algorithms can be classified into 3 main categories

Supervised

Supervised learning algorithms have **labeled** data, which makes it easier to predict the output

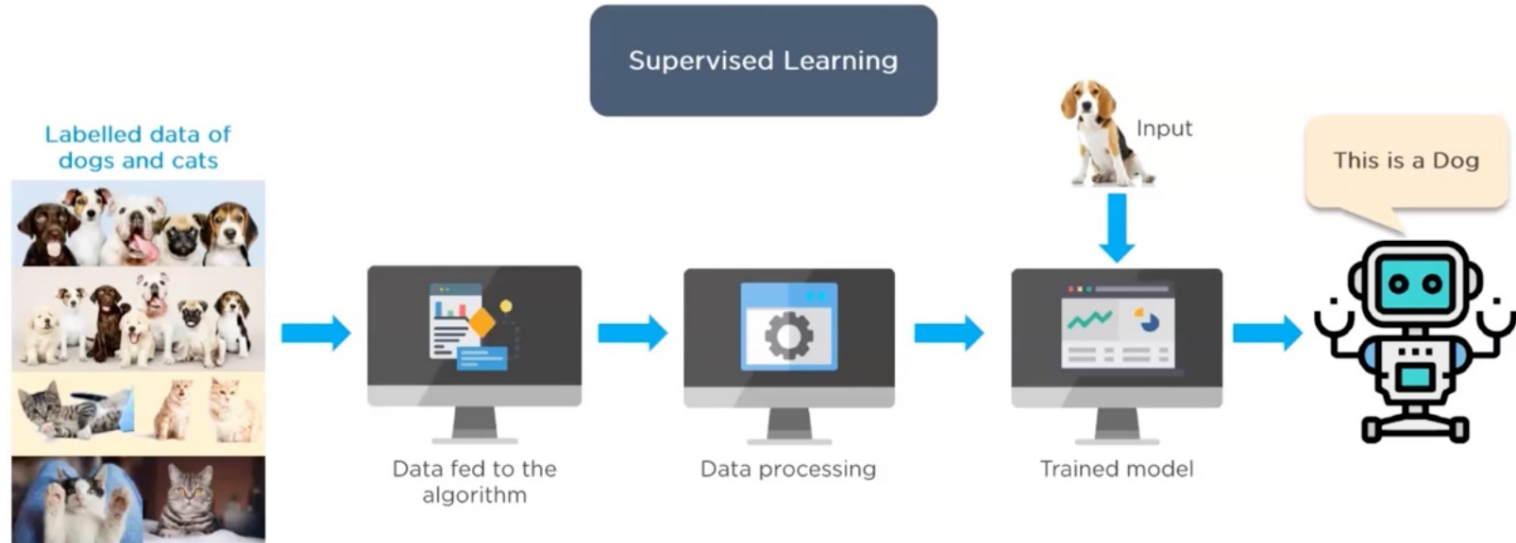
Unsupervised

Unsupervised learning algorithms have **unlabeled** data. It is used to discover patterns from the data

Reinforcement

In Reinforcement learning algorithms, an agent receives a **reward** in the next time step to evaluate its previous action

Machine Learning Algorithms



Machine Learning Algorithms

Supervised Learning

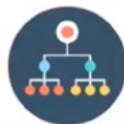
Supervised learning is classified into **Regression** and **Classification**



Linear Regression



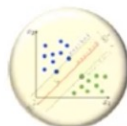
Logistic Regression



Decision Tree



Random Forest



Support Vector Machines



K Nearest Neighbors

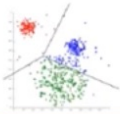


Naïve Bayes

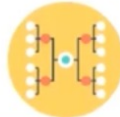
Machine Learning Algorithms

Unsupervised Learning

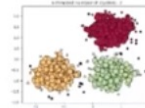
Unsupervised learning is classified into **Clustering** and **Association rules**



K Means Clustering



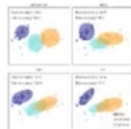
Hierarchical Clustering



DBSCAN



Principal Component Analysis



Gaussian Mixture Model



Hidden Markov Process

Spark MLlib Data Types

MLlib supports local vectors and matrices stored on a single machine, as well as distributed matrices



Local vector

MLlib supports two types of local vectors:
dense and *sparse*

Example: `vector(1.0, 0.0, 3.0)`

dense format: `[1.0, 0.0, 3.0]`

sparse format: `(3, [0, 2], [1.0, 3.0])`

Labeled point

A labeled point is a local vector, either dense or sparse that is associated with a label/response

Example: In binary classification, a label should be either 0 (negative) or 1 (positive)

Spark MLlib Basic Statistics

1

Summary Statistics:

MLlib provides summary statistics for RDD (vector) through the function `colStats()` available in *Statistics* package

`colStats()` returns the column-wise max, min, mean, variance, number of nonzero values, and the total count

```
from pyspark.mllib.stat import Statistics

sc = ... # SparkContext

mat = ... # an RDD of Vectors

# Compute column summary statistics.
summary = Statistics.colStats(mat)
print summary.mean()
print summary.variance()
print summary.numNonzeros()
```

-:-

Spark MLlib Basic Statistics

2

Correlations:

MLlib can perform *correlation* between two *series* of data using the *Statistics* package

Based on the type of input, if there are two RDDs [Double] or an RDD [vector], the output will be a **Double** or the **Correlation Matrix**

```
from pyspark.mllib.stat import Statistics

sc = ... # SparkContext

seriesX = ... # a series
seriesY = ... # must have the same number of partitions and cardinality as seriesX

# Compute the correlation using Pearson's method. Enter "spearman" for Spearman's method. If a
# method is not specified, Pearson's method will be used by default.
print Statistics.corr(seriesX, seriesY, method="pearson")

data = ... # an RDD of Vectors
# calculate the correlation matrix using Pearson's method. Use "spearman" for Spearman's method.
# If a method is not specified, Pearson's method will be used by default.
print Statistics.corr(data, method="pearson")
```

simplilearn

Spark MLlib Basic Statistics

Become A Apache Spark Expert! 

3

Stratified Sampling:

Stratified sampling methods, *sampleByKey* and *sampleByKeyExact*, can be performed on RDD's of key-value pairs

sampleByKeyExact() is currently not supported in Python

The *sampleByKey()* will flip a coin to decide whether an observation will be sampled or not

```
sc = ... # SparkContext  
  
data = ... # an RDD of any key value pairs  
fractions = ... # specify the exact fraction desired from each key as a dictionary  
  
approxSample = data.sampleByKey(False, fractions);
```

Spark MLlib Basic Statistics

4

Random data generation:

Random data generation is used for randomized algorithms, prototyping, and performance testing

Code to generate a random double RDD, whose values follows the standard normal distribution $N(0, 1)$, and then map it to $N(1, 4)$:

```
from pyspark.mllib.random import RandomRDDs

sc = ... # SparkContext

# Generate a random double RDD that contains 1 million i.i.d. values drawn from the
# standard normal distribution `N(0, 1)`, evenly distributed in 10 partitions.
u = RandomRDDs.uniformRDD(sc, 1000000L, 10)
# Apply a transform to get a random double RDD following `N(1, 4)`.
v = u.map(lambda x: 1.0 + 2.0 * x)
```

`RandomRDDs` package provides factory methods to generate random double RDDs or vector RDDs

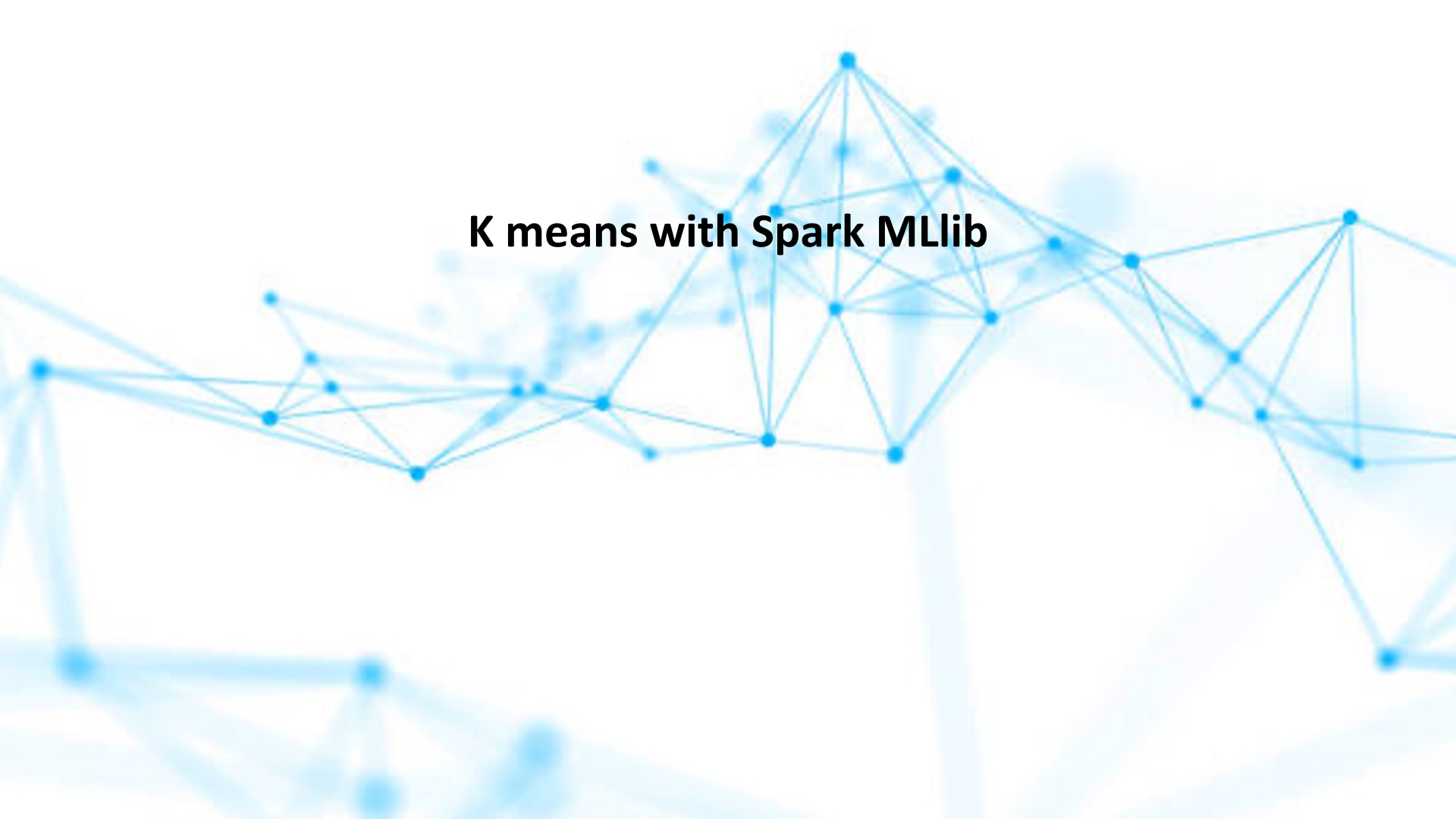
Classification and Regression in MLlib



Spark MLlib supports the following *Classification* and *Regression* algorithms:



simplilearn



K means with Spark MLlib

```
from numpy import array
from math import sqrt

from pyspark.mllib.clustering import KMeans, KMeansModel

# Load and parse the data
data = sc.textFile("data/mllib/kmeans_data.txt")
parsedData = data.map(lambda line: array([float(x) for x in line.split(' ')]))

# Build the model (cluster the data)
clusters = KMeans.train(parsedData, 2, maxIterations=10, initializationMode="random")

# Evaluate clustering by computing Within Set Sum of Squared Errors
def error(point):
    center = clusters.centers[clusters.predict(point)]
    return sqrt(sum([x**2 for x in (point - center)]))

WSSSE = parsedData.map(lambda point: error(point)).reduce(lambda x, y: x + y)
print("Within Set Sum of Squared Error = " + str(WSSSE))

# Save and load model
clusters.save(sc, "target/org/apache/spark/PythonKMeansExample/KMeansModel")
sameModel = KMeansModel.load(sc, "target/org/apache/spark/PythonKMeansExample/KMeansModel")
```

Resources:

<https://towardsdatascience.com/k-means-clustering-using-pyspark-on-big-data-6214beacdc8b>

<https://spark.apache.org/docs/latest/api/python/reference/pyspark.mllib.html>

<https://spark.apache.org/docs/latest/mllib-clustering.html#k-means>

<https://spark.apache.org/docs/latest/api/python/reference/pyspark.mllib.html>